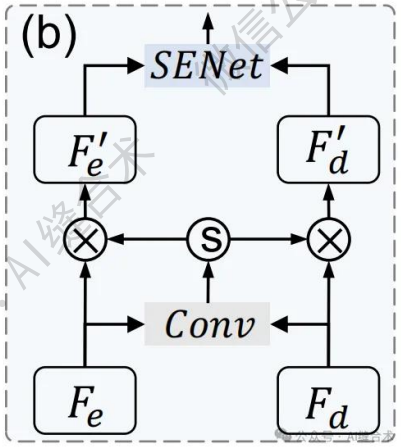
核心速览



论文题目： Partial Decoder Attention Network with Contour Weighted Loss Function for Data-Imbalance Medical Image Segmentation

中文题目： 基于轮廓加权损失函数的偏解码器注意力网络用于数据不平衡的医学图像分割

论文链接： https://arxiv.org/pdf/2601.14338

所属单位： 北京大学健康科学中心医学技术研究所，北京大学国家健康数据科学研究院等

核心速览： 提出一种基于部分解码器注意力网络（PDANet）和轮廓加权损失函数的方法，以解决医学图像分割中的数据不平衡问题，在三个公开数据集上的实验表明该方法优于九种最先进方法，并能提升其他方法的分割性能。

论文内容详细解读：

https://mp.weixin.qq.com/s/5trtJHH7aRlEiWgjemlXxg

```
CWCABlock(
  (conv): Sequential(
    (0): BasicBlock(
      (conv1): Conv3d(32, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1), bias=False)
      (bn1): InstanceNorm3d(16, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
      (relu): ReLU(inplace=True)
      (conv2): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1), bias=False)
      (bn2): InstanceNorm3d(16, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
      (shortcut): Sequential(
        (0): Conv3d(32, 16, kernel_size=(1, 1, 1), stride=(1, 1, 1), bias=False)
        (1): InstanceNorm3d(16, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
        (2): ReLU(inplace=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1), bias=False)
      (bn1): InstanceNorm3d(16, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
      (relu): ReLU(inplace=True)
      (conv2): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1), bias=False)
      (bn2): InstanceNorm3d(16, eps=1e-05, momentum=0.1, affine=False, track_running_stats=False)
      (shortcut): Sequential()
    )
  )
  (weight_conv): Sequential(
    (0): Conv3d(16, 2, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1, 1))
    (1): Softmax(dim=1)
  )
  (channel_attention): ChannelAttention(
    (avg_pool): AdaptiveAvgPool3d(output_size=1)
    (max_pool): AdaptiveMaxPool3d(output_size=1)
    (fc): Sequential(
      (0): Linear(in_features=32, out_features=8, bias=False)
      (1): ReLU(inplace=True)
      (2): Linear(in_features=8, out_features=32, bias=False)
    )
    (sigmoid): Sigmoid()
  )
  (se): SELayer(
    (avg_pool): AdaptiveAvgPool3d(output_size=1)
    (conv): Sequential(
      (0): Conv3d(32, 8, kernel_size=(1, 1, 1), stride=(1, 1, 1))
      (1): ReLU(inplace=True)
      (2): Conv3d(8, 32, kernel_size=(1, 1, 1), stride=(1, 1, 1))
      (3): Sigmoid()
    )
  )
)
```



```
input_tensor1_shape  : torch.Size([1, 16, 128, 128, 128])
input_tensor2_shape  : torch.Size([1, 16, 128, 128, 128])
output_tensor_shape  : torch.Size([1, 32, 128, 128, 128])

哔哩哔哩/微信公众号：AI缝合术，独家整理！
```